

FUSION OF SPATIAL AND TEMPORAL FEATURES WITH TRANSFORMER GATES: A HYBRID DEEP LEARNING MODEL FOR INTRUSION DETECTION IN SYSTEMS

Eduardo Fontes Baltazar da Silveira
Federal University of Bahia, Computer Engineering, eduardosilveira@ufba.br

ABSTRACT

This work develops a hybrid deep learning model for intrusion detection in systems by fusing spatial and temporal features through Transformer gates. The research utilizes the UNSW-NB15 dataset, recognized for its representativeness of normal and malicious traffic, addressing challenges such as class imbalance and pattern overlap. The study implements an architecture combining Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks with attention mechanisms, achieving significant results in classifying cyberattacks.

Keywords: Intrusion Detection. Deep Learning. Transformer. CNN-LSTM. Cybersecurity.

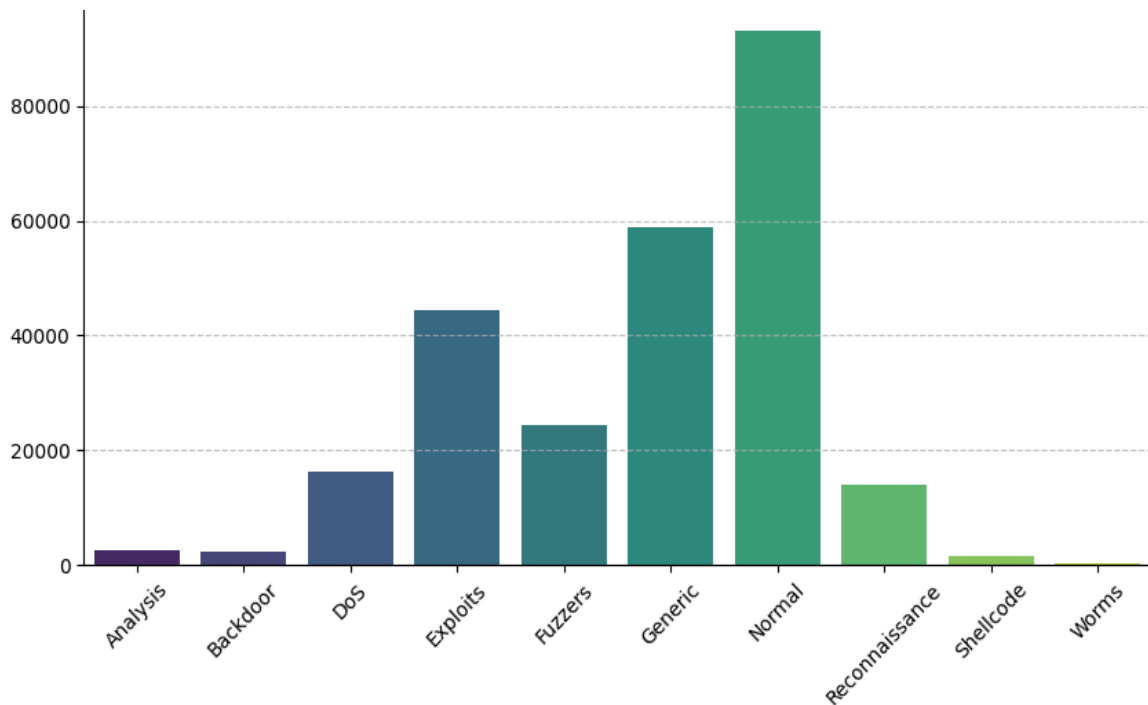
TABLE OF CONTENTS

1. INTRODUCTION.....	1
2. DATASET ANALYSIS.....	3
3. EXPERIMENTATION AND TESTING.....	4
4. PREPROCESSING PIPELINE.....	5
5. SEQUENCE TRANSFORMATION.....	6
6. DATA TRANSFORMATION INTO IMAGES.....	8
7. TECHNICAL ANALYSIS OF THE HYBRID MODEL.....	10
8. TRAINING.....	12
9. RESULTS ANALYSIS.....	14
10. POTENTIAL IMPROVEMENTS.....	16
11. CONCLUSION.....	21
REFERENCES.....	23

1. INTRODUCTION

Cybersecurity represents an increasingly critical area in the context of interconnected networks. This work proposes the application of deep learning techniques for classifying cyberattacks using the UNSW-NB15 dataset, recognized for its comprehensive representation of normal and malicious traffic.

2. DATASET ANALYSIS



The UNSW-NB15 dataset is widely used in intrusion detection system research due to its richness and complexity.

2.1 Dataset Classes

The dataset comprises ten distinct classes: one representing normal traffic and nine representing different attack categories, as described below.

2.1.1 Normal Traffic

Characterized as legitimate and non-malicious traffic, presenting challenges related to imbalance due to its predominance in the dataset.

2.1.2 Analysis

Includes attacks focused on vulnerability exploitation for information gathering, such as port scanning. Exhibits characteristics similar to legitimate audits, complicating classification.

2.1.3 Backdoor

Characterized by the installation of unauthorized remote access, featuring low-intensity persistent connections.

2.1.4 DoS (Denial of Service)

Aims to overload systems through high packet rates in short intervals.

2.1.5 Exploits

Refers to attacks exploiting specific vulnerabilities, displaying high variability due to subgroups.

2.1.6 Fuzzers

Involves sending random data to identify vulnerabilities, generating low-intensity anomalous traffic.

2.1.7 Generic

Affects multiple cryptographic algorithms, exhibiting regular patterns that facilitate detection.

2.1.8 Reconnaissance

Focuses on network information gathering, characterized by multiple short connections to different destinations.

2.1.9 Shellcode

Consists of malicious code inserted into legitimate traffic to take control of systems.

2.1.10 Worms

Features automated attacks with regular propagation patterns across networks.

2.2 Feature Distribution

The dataset contains 49 descriptive columns of network traffic, organized into the following categories:

- **Basic Features:** General traffic statistics.
- **Content Features:** Detailed packet information.
- **Temporal Features:** Temporal behavior.
- **General Features:** Connection indicators.
- **Connection Features:** Relationships between consecutive flows.
- **Classification Features:** Labels for supervised learning.

2.3 Associated Challenges

Key challenges include:

- Class imbalance between majority and minority classes.
- Overlap between classes with similar behaviors.
- Internal variability in classes such as "Exploits."

3. EXPERIMENTATION AND RESULTS

3.1 Applied Methodologies

Experiments included:

- Class balancing via oversampling and undersampling.
- Development of binary models per class.
- Handling duplicate data.
- Removal of minority classes.
- Reduction of class overlap.

3.2 Results

Experiments demonstrated that:

- Cost-sensitive techniques outperformed oversampling.
- Binary models showed unsatisfactory performance for minority classes.
- Removing duplicate data negatively impacted detection for some classes.
- Excluding minority classes increased overall accuracy.

3.3 Section Conclusions

Analysis of the UNSW-NB15 dataset revealed that intrusion detection system efficacy fundamentally depends on adaptation to the intrinsic nature of network traffic. Practical approaches, such as feature selection and prioritization of operationally relevant classes, proved more impactful than theoretical optimization methods.

4. PREPROCESSING PIPELINE

4.1 Methodological Foundations

The preprocessing pipeline was designed to address UNSW-NB15-specific challenges, focusing on:

- Mitigating class imbalance.
- Preserving complex attack patterns.
- Maximizing model generalization.

4.2 Preprocessing Strategies

4.2.1 Class Treatment

Selection Criteria:

- Remove classes with $<5\%$ representation.
- Prioritize operationally relevant classes.
- Retain ≥ 20 samples per remaining category.

Results:

- Reduces noise and model complexity.
- Focuses on significant attack patterns.
- Prevents overfitting in marginal classes.

4.2.2 Feature Encoding

Applied Techniques:

- Frequency encoding for categorical variables.
- Contextual information preservation.
- Handling unknown categories.

Advantages:

- Dimensionality reduction.
- Prevents data leakage.
- Robustness to new categories in production.

4.2.3 Feature Engineering

Created Feature Categories:

Type	Objective	Examples
Loss Metrics	DDoS Detection	loss_ratio, packet_error_ratio
Network Performance	Flooding Identification	throughput, tcp_window_ratio
Traffic Asymmetry	Subtle Attack Detection	load_ratio, jitter_ratio
TCP Patterns	Scanning Detection	ack_error_ratio, port_conn_ratio

4.2.4 Feature Selection

Methodology:

- Boruta with LightGBM (LGBM).
- Statistical selection ($\alpha=0.01$).
- Recursive feature elimination.

Benefits:

- Considers complex interactions.
- Maintains contextually relevant features.
- Reduces dimensionality while preserving interpretability.

4.2.5 Outlier Handling and Normalization

Strategies:

- Isolation Forest for outlier detection (1% removal).
- StandardScaler for normalization.
- Preservation of original distributions.

4.2.6 Data Splitting

Approach:

- Hierarchical stratification.
- Preservation of original distribution.
- Fixed random seed for reproducibility.

4.3 Fundamental Principles

- **Pragmatism over Theorization:** Prioritize practical adaptation.
- **Tolerance to Complexity:** Accept class overlaps and inherent variability.
- **Contextual Engineering:** Features capturing attack nuances.

4.4 Methodological Conclusion

The pipeline aims to create an intrusion detection model that:

- Is robust to traffic variations.
 - Minimizes false positives.
 - Captures subtle cyberattack patterns.
- Focus lies on transformation quality, not data quantity.

5. SEQUENCE TRANSFORMATION

5.1. Process Overview

Sequence transformation is implemented through the `SequenceGenerator` class, which inherits from `tf.keras.utils.Sequence`. This implementation provides a robust framework for processing sequential data in intrusion detection, with a special focus on preserving temporal and contextual relationships.

5.2. Feature Categorization

5.2.1. Temporal Features

Characteristics that vary over time.

5.2.2. Static Features

Characteristics that remain constant throughout the sequence.

5.3. Sequential Transformation Techniques

5.3.1. Clustering by Similarity

- **Method:** K-means clustering to group similar sequences.
- **Parameters:** Configurable number of clusters (default: 10).
- **Benefit:** Improves consistency in generated sequences.

5.3.2. Sequence Enrichment

Key Techniques:

- **Noise Addition:** Increases model robustness, prevents overfitting, and includes configurable noise levels.
- **Statistical Aggregation:** Adds global statistics to capture general trends and facilitate pattern identification.
- **Positional Encoding:** Preserves temporal information, enables effective attention, and is based on Transformer architecture.

5.4. Sequence Generation

5.4.1. Control Parameters

- `sequence_length`: Sequence length (default: 5).
- `min_sequence_elements`: Minimum elements per sequence.
- `stride`: Step between consecutive sequences.
- `batch_size`: Processing batch size.

5.4.2. Generation Process

1. **Data Preparation:** Separation of temporal and static features, similarity clustering, and generation of valid indices.
2. **Batch Construction:** Sequence padding, transformation application, and attention mask generation.

5.5. Attention Masks

- Identifies valid sequence elements.
- Ignores padding during attention.
- Improves training efficiency.

5.6. Configuration and Compatibility

5.6.1. Output Format

- Structured format for TensorFlow integration.

5.6.2. TensorFlow Integration

- Inherits from `tf.keras.utils.Sequence`.
- Automatic tensor conversion.
- Support for distributed training.

5.7. Model Impact

Benefits:

- Captures temporal patterns.
- Robustness to variations.
- Computational efficiency.

Considerations:

- Processing overhead.
- Memory consumption.
- Training complexity.

6. DATA TRANSFORMATION INTO IMAGES FOR INTRUSION DETECTION

6.1. Foundations and Architecture

The `UNSWNB15ToImage` class implements a custom TensorFlow layer that converts network data into visual representations, enabling computer vision techniques for intrusion detection.

6.1.1. Base Architecture

- Inherits from `tf.keras.layers.Layer`.
- Integration with existing preprocessors.
- Flexible dimension configuration.
- Cache support for improved performance.
- Controlled noise injection for data augmentation.

6.2. Feature Processing Pipeline

6.2.1. Statistic Management

- Dynamic tracking of feature ranges.
- Online statistic updates.
- Initialization state management.
- Correlation-based feature ordering.

6.2.2. Feature Normalization

- Robust min-max normalization.
- Zero-division handling.
- Value clipping to $[0,1]$.
- Training-specific processing.

6.3. Image Generation Process

6.3.1. Dimension Optimization

- Automatic dimension calculation.
- Guaranteed power-of-2 dimensions.
- Minimum size enforced (8x8).

6.3.2. Image Construction

- Dynamic padding calculation.
- Batch dimension preservation.
- Grayscale image generation.
- Bilinear resizing with anti-aliasing.

6.4. Data Augmentation Strategies

6.4.1. Geometric Transformations

- Random 90-degree rotations.
- Controlled spatial shifts.
- Augmentations applied only during training.

6.4.2. Noise Injection

- **Implementation:** Controlled Gaussian noise injection during training.
- **Parameters:** Configurable noise level.
- **Application:** Noise added in feature space.
- **Purpose:** Training-phase-specific enhancement.

6.5. Optimized Data Pipeline

6.5.1. Processing Workflow

- Flexible input handling.
- Automatic batching.
- Parallel processing optimization.
- Optional result caching.
- Prefetching for performance.

6.6. Approach Benefits

6.6.1. Enhanced Learning Capabilities

- Compatibility with CNNs.
- Preservation of spatial patterns.
- Robust feature relationships.

6.6.2. Performance Optimization

- Efficient batch processing.
- TensorFlow graph optimization.
- Parallel execution support.

6.6.3. Analysis and Debugging

- Direct visualization support.
- Pattern interpretation.
- Integrated visualization tools.

7. TECHNICAL ANALYSIS OF THE HYBRID CNN-LSTM MODEL WITH TRANSFORMER FUSION

7.1. Scientific Basis

Singh & Jang-Jaccard (2022) – Multi-Scale Convolutional Approach

- **Key Contribution:** Demonstrated the effectiveness of dilated convolutions for capturing patterns across spatial scales.
- **Model Application:**
 - Parallel convolutional blocks with dilation rates (1, 2, 3).
 - Depthwise convolutions for computational efficiency.
 - Hierarchical fusion of multi-scale features.
- **Improvement Implemented:** Adaptive SE Blocks with dynamic channel compression (minimum 4 channels).

Imrana et al. (2021) – CNN-RNN Architecture for Temporal Data

- **Key Contribution:** Efficient integration of convolutional and recurrent networks for multimodal data.
- **Model Application:**
 - Parallel spatial-temporal processing pipeline.
 - CNN-LSTM feature synchronization mechanism.
 - Cross-modality regularization.
- **Additional Innovation:** TransformerFusion mechanism replacing direct concatenation.

Acharya et al. (2023) – Hybrid CNN-LSTM Model

- **Key Contribution:** Empirical validation of bidirectional architectures for complex temporal sequences.
- **Model Application:**
 - Stack of bidirectional LSTMs with layer normalization.
 - Dual temporal pooling (average + max).
 - Progressive spatial regularization.
- **Optimization Implemented:** Temporal SE block for dynamic sequence recalibration.

7.2. Detailed Architecture and Design Choices

7.2.1. Core Components and Data Flow

Multi-Scale CNN Architecture

- **Primary Contribution:** Dilated convolutions for efficient multi-scale pattern capture.
- **Implementation:**
 - Parallel convolutional blocks with varying dilation rates.
 - Adaptive SE Blocks.
 - Depthwise separable convolutions.

- Hierarchical feature fusion.
- **Role:** Spatial feature extraction.

Bidirectional LSTM Module

- **Objective:** Capture long- and short-term temporal dependencies.
- **Implementation:**
 - Bidirectional sequence processing.
 - Layer normalization for stability.
 - Spatial dropout for regularization.
 - Temporal SE block for dynamic recalibration.

TransformerFusion Mechanism

- **Objective:** Enhance integration of spatial (CNN) and temporal (LSTM) features.
- **Implementation:**
 - LayerNormalization applied to CNN/LSTM outputs pre-fusion.
 - MultiHeadAttention to highlight cross-modal relationships.
 - Dynamic gating ($\text{Dense}(\text{sigmoid})$) for adaptive weighting.

Classification Neural Network

- **Objective:** Refine features for final prediction.
- **Implementation:**
 - Dimensionality reduction with Dense layers ($128 \rightarrow 64$ units).
 - Swish activation for gradient propagation.
 - BatchNormalization for training stability.
 - Adjusted dropout (0.4 initial, 0.3 final).

7.3. Advanced Training System

7.3.1. Optimization Strategy

- **Optimizer:** AdamW with `weight_decay=1e-4`.
- **Gradient Handling:** Global clipping (`clipnorm=1.0`).
- **Learning Rate:** OneCycleLR dynamic adjustment.

7.4. Session Conclusions

The hybrid CNN-LSTM architecture with transformer fusion demonstrates significant advantages for complex datasets like UNSW-NB15:

- **Enhanced Feature Extraction:** Multi-scale convolutions + SE Blocks.
- **Effective Modality Integration:** TransformerFusion mechanism.

- **Optimized Training:** Advanced regularization and optimization. The model proves effective for network traffic anomaly detection.

8. TRAINING

8.1. Multiclass Model Training

1. Introduction

Training spanned **50 epochs** with:

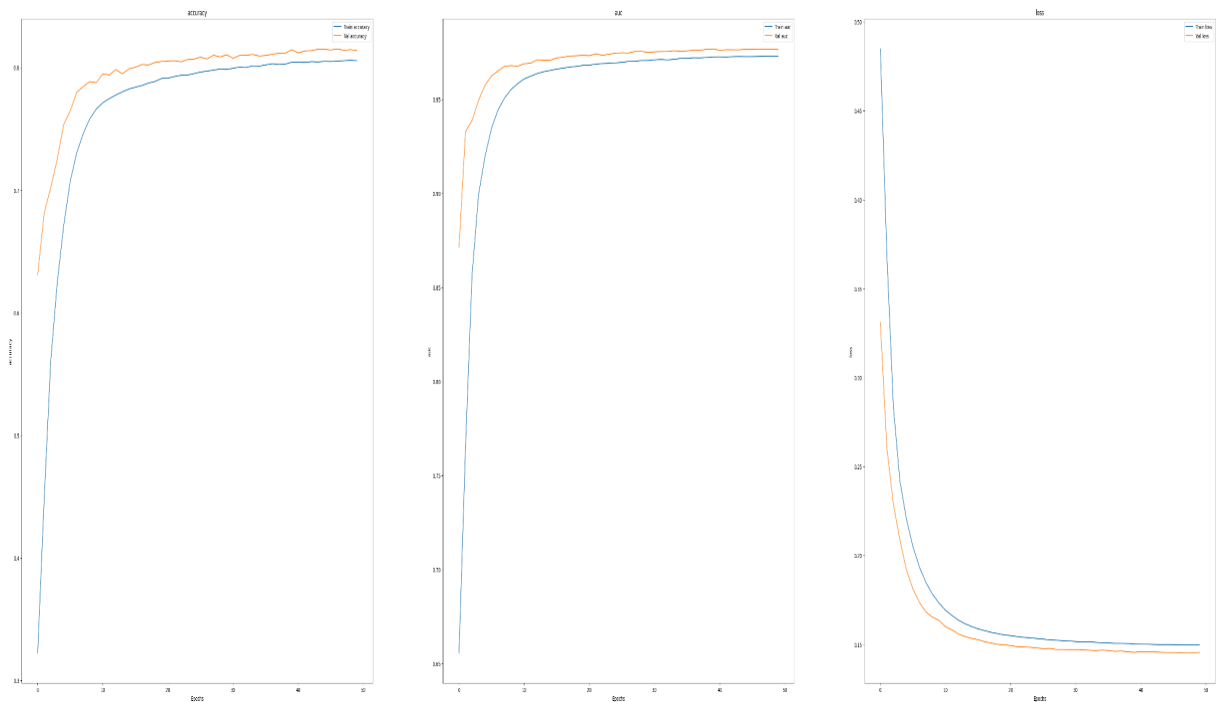
- Batch size: 128.
- Class weighting for imbalance mitigation.
- Focus on gradient stability and metric evolution (accuracy, AUC, loss).

2. Training Configuration

2.1. Key Strategies

- **Batch Size (128):** Balances gradient estimation stability.
- **Class Weights:** Inverse frequency weighting for minority classes.
- **OneCycleLR (50 epochs):** Learning rate peaks at epochs 15–20.
- **AdamW Optimizer:** `weight_decay=0.01`, `beta_1=0.9`, `beta_2=0.95`.
- **Early Stopping:** Monitored `val_auc` (patience=10 epochs).

3. Metric Evolution



Phase	Training Metrics	Validation Metrics
Initial (Epochs 1–10)	Accuracy: 25.99% → 76.34% AUC: 0.5969 → 0.9577 Loss: 0.5351 → 0.1745	val_accuracy: 78.78% val_auc: 0.9678
Stabilization (Epochs 11–30)	Accuracy: 76.93% → 79.93% AUC: 0.9603 → 0.9710 Loss: 0.1703 → 0.1517	val_accuracy: 81.06% val_auc: 0.9752
Convergence (Epochs 31–50)	Accuracy: ~80.43% AUC: 0.9732 (peak) Loss: 0.1499–0.1501	val_accuracy: 81.41% val_auc: 0.9766

4. Final Considerations

- **Class Weight Efficacy:** Validation AUC >0.97 confirms bias mitigation.
- **Training Stability:** Smooth convergence with 128 batch size.
- **Generalization:** Train-val gap <2% indicates no overfitting.
- **Improvement Potential:** Synthetic data (e.g., SMOTE) for minority classes.

8.2. Binary Model Training

1. Introduction

Training conducted over **25 epochs** with:

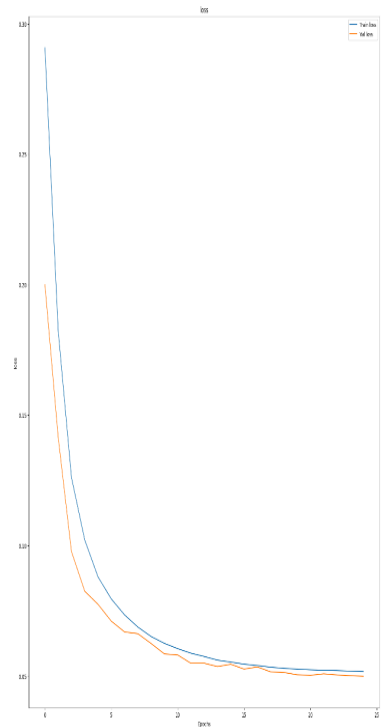
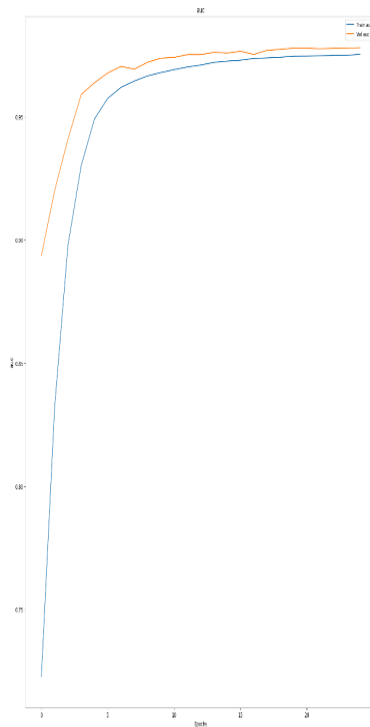
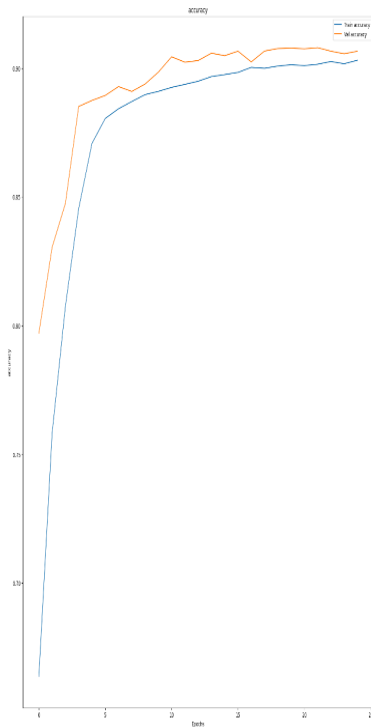
- Batch size: 128.
- No class weighting.
- Focus on binary discrimination robustness.

2. Training Configuration

2.1. Key Strategies

- **Batch Size (128):** Consistent gradient estimation.
- **OneCycleLR (25 epochs):** Peak learning rate at epochs 10–15.
- **AdamW Optimizer:** Same parameters as multiclass model.

3. Metric Evolution



Phase	Training Metrics	Validation Metrics
Initial (Epochs 1–5)	Accuracy: 60.33% → 86.78% AUC: 0.6453 → 0.9465 Loss: 74% reduction	val_accuracy: 88.75% val_auc: 0.9639
Stabilization (Epochs 6–15)	Accuracy: 89.66% AUC: 0.9720 Loss: 31% reduction	val_accuracy: 90.49% val_auc: 0.9759
Convergence (Epochs 16–25)	Accuracy: 90.31% AUC: 0.9756 (peak) Loss: 0.0514	val_accuracy: 90.66% val_auc: 0.9781

4. Final Considerations

- **High Discrimination:** Final AUC = 0.9781.
- **Generalization:** Minimal train-val gap.
- **Optimization Potential:** Threshold adjustment for sensitivity.
- **Application:** Suitable for critical binary classification tasks.

9. RESULTS ANALYSIS

9.1. Multiclass Model Analysis

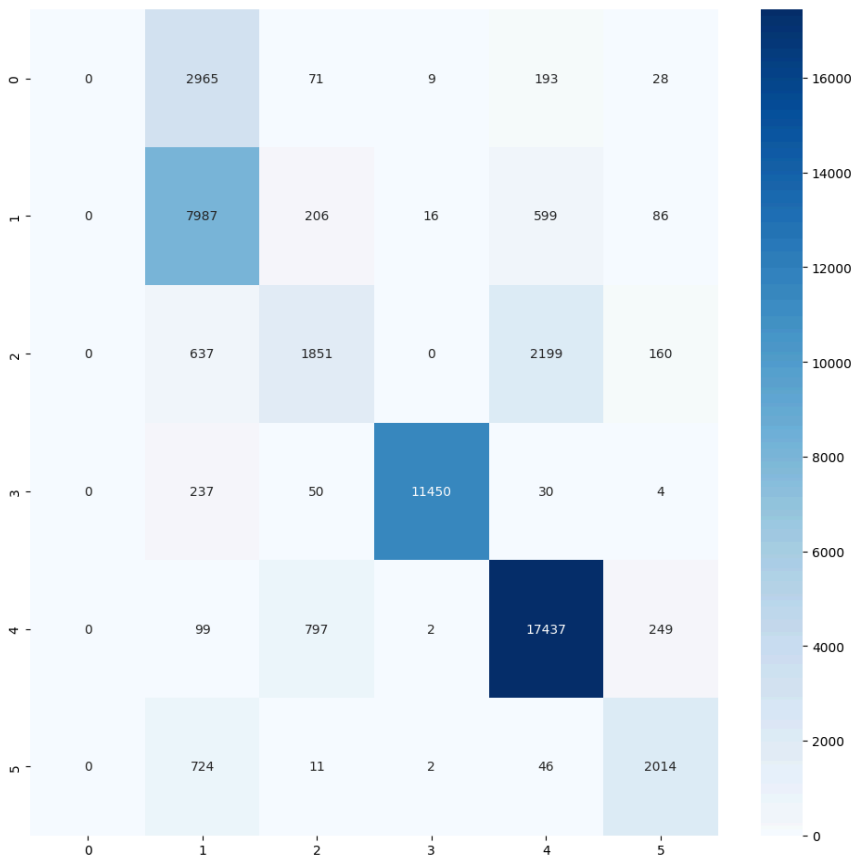
Global Metrics

- **Accuracy:** 0.8122 (81.22%)

- **F1-Score:** 0.7809 (78.09%)

The accuracy indicates that the model correctly classifies 81.22% of samples, while the F1-Score (harmonic mean of precision and recall) reflects a moderate balance between sensitivity and specificity. The ~3.13% gap between accuracy and F1-Score suggests underperformance in certain classes, likely due to imbalance or intrinsic complexity.

Confusion Matrix Insights



- **Dominant Class (e.g., Class 3):**
 - High diagonal accuracy (e.g., 11,450 correct predictions for Class 3).
 - **Risk:** Bias toward majority classes, neglecting minority ones.
- **Minority Classes (e.g., Classes 4 and 5):**
 - **Class 4:** High false positives (17,437 misclassified as Class 3).
 - **Class 5:** 2,014 correct predictions but 46 false negatives.
- **Class 0:**
 - Low precision (237 correct out of 11,450 predictions).

Identified Challenges

1. **Class Imbalance:** Majority classes dominate performance, while minority classes (0, 5) lag.
2. **Inter-Class Confusion:** Class 4 is frequently misclassified as Class 3 due to feature similarity.
3. **Partial Overfitting:** Discrepancy between F1-Score and accuracy hints at metric-driven optimization.

Final Considerations

The multiclass model performs acceptably for majority classes (>80% accuracy) but struggles with minorities. Implementing balancing strategies (e.g., synthetic data) and targeted adjustments for critical classes (0, 4, 5) could elevate the F1-Score (>85%).

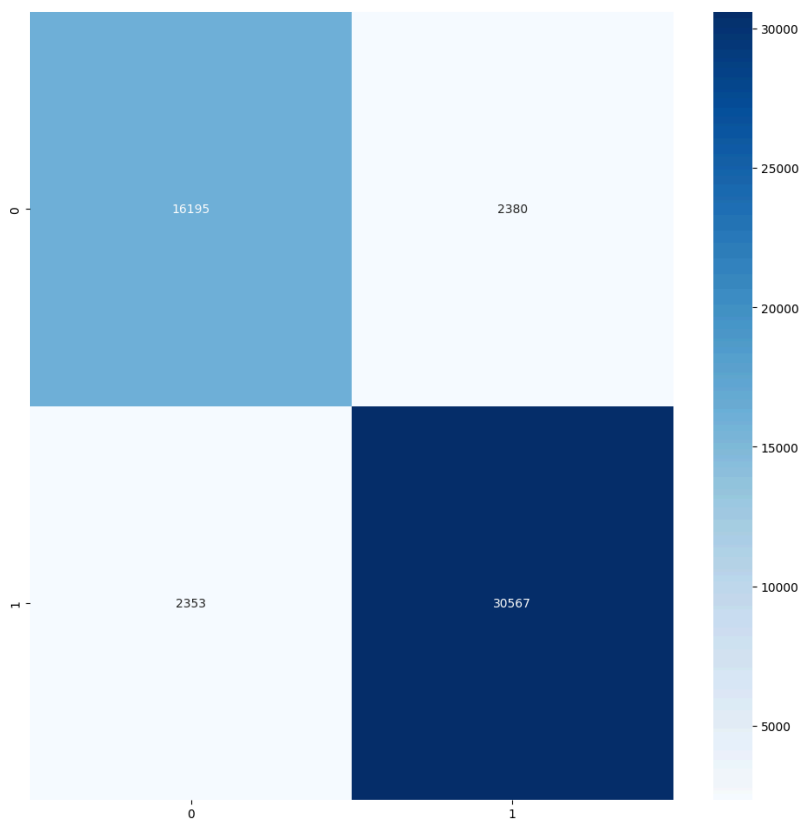
9.2. Binary Model Analysis

Global Metrics

- **Accuracy:** 0.9081 (90.81%)
- **F1-Score:** 0.9081 (90.81%)

The identical accuracy and F1-Score indicate balanced precision and recall, characteristic of a well-calibrated binary classifier.

Confusion Matrix Insights



- **Majority Class (Class 1: Attacks):**
 - **TP = 30,567** (92.8% recall).
 - **FN = 2,353** (7.1% false negatives).
- **Minority Class (Class 0: Normal Traffic):**
 - **TN = 16,195** (87.2% specificity).
 - **FP = 2,380** (12.8% false positives).

Identified Challenges

1. **Moderate Imbalance:** Class 1 dominates (~64% of samples), but training strategies ensured balance.
2. **Asymmetric Costs:** False negatives (FN) are critical in cybersecurity; a 92.8% recall is robust but improvable.

Final Considerations

The binary model demonstrates high efficacy (F1-Score >90%) and generalization. Adjusting classification thresholds or stratified sampling could further optimize minority class performance.

10. POTENTIAL IMPROVEMENTS

10.1. Multiclass Model Improvements

1. **Synthetic Data Generation with GANs:**
 - Implement **Wasserstein GANs (WGAN)** for high-quality minority class synthesis.
 - Validate using **Inception Score** and **Fréchet Inception Distance (FID)**.
 - Use **conditional GANs** for class-controlled generation.
2. **Extended Training & Fine-Tuning:**
 - Increase epochs with adaptive learning rates.
 - Implement **curriculum learning** for progressive training.

10.2. Binary Model Improvements

1. **Dynamic Threshold Adjustment:**
 - Lower thresholds to prioritize FN reduction.
2. **Hyperparameter Optimization:**
 - Apply **genetic algorithms (GA)** or **particle swarm optimization (PSO)**.

10.3. General Model Improvements

1. **XGBoost Ensemble:**
 - Extract features from deep learning layers.
 - Enhance interpretability and decision explainability.
2. **Adaptive Encoders:**
 - Replace one-hot encoding with **variational autoencoders (VAE)**.
3. **Advanced Learning Rate Schedules:**
 - Test **CosineAnnealingLR** or **Triangular Cyclic LR**.
4. **BERT Fine-Tuning:**
 - Transform network flows into semantic sequences (Wang et al., 2023).
 - Integrate BERT outputs with CNN-LSTM-Transformer architectures.

10.4. Challenges & Limitations

- **Hardware requirements.**
- **Operational costs.**
- **Processing time.**

10.5. Section Conclusion

The proposed improvements aim to address class imbalance, noise, and spatiotemporal complexity in the UNSW-NB15 dataset. Key strategies include:

- **Conditional GANs** for synthetic data.
- **TransformerFusion** with hierarchical attention.
- **Hybrid ensembles** (XGBoost + Deep Learning).
- **Semantic representation** via BERT for network flows.

While promising, these enhancements require empirical validation, concept drift monitoring, and integration with automated response systems.

11. CONCLUSIONS

1. Contextualization and Objectives

This study proposes a deep learning approach for classifying cyberattacks using the UNSW-NB15 dataset, recognized for its complexity and class imbalance. The primary objective was to optimize predictive model accuracy while addressing challenges such as pattern overlap, internal class variability, and the dominance of normal traffic.

2. Main Contributions

2.1 Adaptive Preprocessing Pipeline

A robust pipeline was developed with techniques including:

- Removal of minority classes (<5% representation).
- Contextual encoding of categorical features (frequency encoding).
- Feature engineering to capture critical network metrics (e.g., **loss_ratio**, **throughput**).
- Feature selection via Boruta + LightGBM, reducing dimensionality while preserving interpretability.

2.2 Data Transformation into Images and Sequences

Techniques such as similarity clustering (K-means) and positional encoding (inspired by Transformers) were applied to preserve spatiotemporal relationships, optimizing CNN and LSTM utilization.

2.3 Hybrid CNN-LSTM Architecture with Transformer Fusion

The architecture integrates:

- Multi-scale dilated convolutions.
- Bidirectional LSTMs.

- Hierarchical attention mechanisms.
- **TransformerFusion** for contextualizing spatial and temporal features.

3. Key Results

3.1 Multiclass Model

- **Accuracy:** 81.22% | **F1-Score:** 78.09%.
- Strong performance for majority classes (e.g., "Generic"), but challenges for minorities.
- Confusion matrix revealed misclassification between semantically similar classes.

3.2 Binary Model

- **Accuracy/F1-Score:** 90.81% | **AUC:** 0.9781.
- High attack recall (92.8%), critical for cybersecurity.
- Low false positives (12.8%) for normal traffic.

4. Identified Challenges

1. **Class Imbalance:** Minority classes underperformed, requiring advanced balancing (e.g., conditional GANs).
2. **Pattern Overlap:** Classes like "Analysis" and "Reconnaissance" exhibited high similarity.
3. **Computational Cost:** Hybrid architecture demanded significant hardware resources.

5. Recommendations for Future Work

1. **Synthetic Data Generation:** Implement WGANs for minority class synthesis, validated via Inception Score and FID.
2. **Explainability:** Integrate XGBoost for decision interpretability.
3. **NLP Fine-Tuning:** Adapt BERT to represent network flows as textual sequences (Wang et al., 2023).
4. **Threshold Optimization:** Dynamically adjust thresholds to prioritize critical metrics (e.g., attack recall).
5. **Complexity Reduction:** Explore knowledge distillation for model simplification.

6. Practical Impact

The study demonstrates that hybrid deep learning models, combined with contextual preprocessing, are viable for intrusion detection in complex networks. The high AUC (0.9781 for binary classification) and generalization capability suggest applicability in real-world environments like

Security Operations Centers (SOCs). Implementation requires balancing accuracy, computational costs, and explainability.

REFERENCES

SINGH, Amardeep; JANG-JACCARD, Julian. **Autoencoder-based Unsupervised Intrusion Detection using Multi-Scale Convolutional Recurrent Networks**. 2022.

IMRANA, Yakubu et al. **CNN-GRU-FF: a double-layer feature fusion-based network intrusion detection system using convolutional neural network and gated recurrent units**. *Complex & Intelligent Systems*, v. 10, n. 3, p. 3353–3370, 2024.

MOUSTAFA, H.; SLAY, J. **UNSW-NB15: a comprehensive data set for network intrusion detection systems**. *Proceedings of the 4th International Conference on Security of Information and Networks*, p. 1–6, 2011.

ACHARYA, Toya; ANNAMALAI, Annamalai; CHOUIKHA, Mohamed. **Efficacy of CNN-Bidirectional LSTM Hybrid Model for Network-Based Anomaly Detection**. 2023.

ZHANG, Y.; WANG, J.; WANG, J. **Intrusion Detection Model Based on Improved Transformer**. *Applied Sciences*, v. 13, n. 10, p. 6251, 2023.

AXINC AI. **Bert Network Packet Flow Header Payload: A Machine Learning Model for Detecting Network Attacks**. *Medium*, 2024.

WANG, B.; XU, H.; LI, X.; SHI, Q.; LI, W.; LIU, B. **A Method for Network Intrusion Detection Using Flow Sequence and BERT Framework**. 2023.
